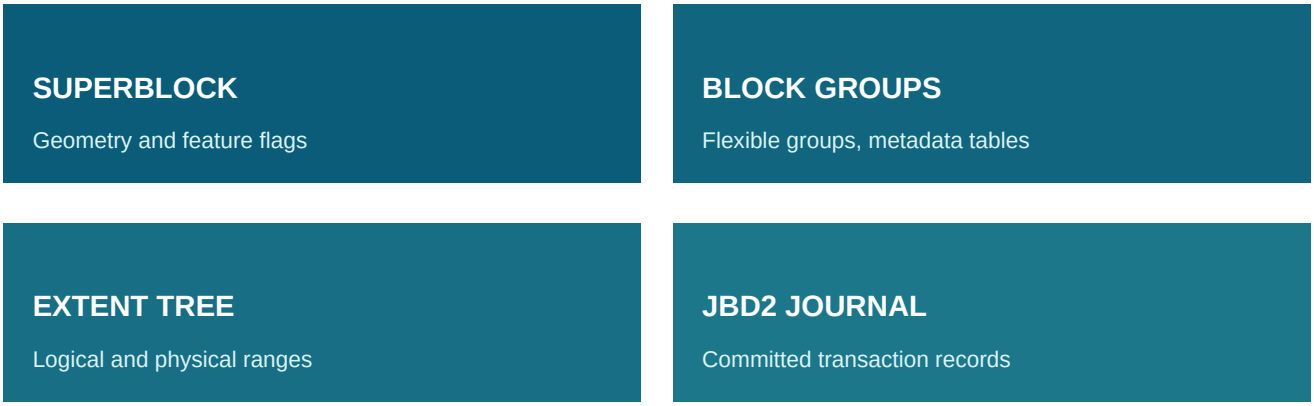
EXT4 - fourth extended filesystem

HISTORY / ORIGINAL DESIGN / DISK FORMAT

The Linux ext-family format evolved for large filesystems, extent mapping, delayed allocation, large directories and checksum-protected metadata.

This is a technical reference to the upstream filesystem's historical design, not an installer manual. The format's on-media semantics are distinct from the independent InfiltratorOS module, its APIs and any Linux kernel adapter.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

EXT4 - fourth extended filesystem

HISTORICAL CHRONOLOGY

Ext4 work began as an evolution of ext3 in the 2000s, initially developed through an ext4dev stage and accepted as a stable mainline filesystem in Linux 2.6.28 in 2008. Its purpose was to escape ext3's limitations in filesystem and file size while improving fragmented-file behaviour and allocation efficiency. The ext2 to ext3 to ext4 progression retained a significant degree of on-disk compatibility under explicit feature flags.

Ext4 became a mainstream Linux default on desktops, servers and embedded systems. Over time the format grew features including metadata checksums, 64-bit block numbering, bigalloc, inline data and encryption support. These cannot be understood simply from the volume's name: the compatible, readonly-compatible and incompatible feature masks determine what an implementation may safely read or write.

EXT4 - fourth extended filesystem

ORIGINAL ON-DISK ARCHITECTURE

The main superblock is at offset 1024 bytes from the start of the filesystem. Block group descriptors identify inode tables, allocation bitmaps and group metadata. Flex block groups can group metadata from multiple ordinary groups together for locality. Most ext4 on-disk fields are little-endian; the JBD2 journal's structures use big-endian encoding, an important distinction in independent parsers.

Extents replace long lists of individual block pointers for many files. An extent tree has a header recording depth and entry counts, with index nodes at intermediate levels and extent leaves mapping logical file blocks to physical ranges. A small root fits inside the inode. Data checksums are not universal, and extent presence does not imply that every allocated range is initialised.

EXT4 - fourth extended filesystem

ALLOCATION AND DIRECTORIES

Delayed allocation postpones physical block assignment until writeback, allowing the multiblock allocator to choose larger contiguous runs. This reduces immediate fragmentation while creating recovery obligations around data that has not yet been placed on the disk. Group locality, preallocation and free-space summaries guide the allocator. Physical contiguity is a policy goal rather than a structural promise.

Directories reference inodes through variable-length records. For larger directories, HTree hashed indexing reduces search cost; inline data and indexed-directory features change exact record locations. Inodes hold ownership, mode, timestamps, size, flags and the mapping root. Extents, block-size selection and 64-bit feature flags must be interpreted consistently across metadata and allocation operations.

EXT4 - fourth extended filesystem

INTEGRITY AND RECOVERY

The JBD2 journal provides transactional metadata recovery, commonly using ordered-data mode. Ordered-data journalling is not a promise to journal every byte of user content; it constrains ordering relative to metadata transactions. A completed commit is meaningful only when storage flush and barrier guarantees are honoured. Metadata checksum features can detect damage to many critical structures when enabled.

After an unclean shutdown, the journal may replay committed transactions. e2fsck checks allocation consistency, inode connectivity, reference counts and other issues not resolved by journal replay. A missing journal checksum or unexpected feature flag must not be silently ignored. An independent recovery tool must not modify a concurrently mounted ext4 volume.

EXT4 - fourth extended filesystem

DESIGN RATIONALE AND LIMITATIONS

The extent model and delayed allocator improve large sequential workloads and reduce the penalty of fragmented indirect-block maps. Flex groups can improve placement of inode tables and file data. These advantages add complexity: an implementation must handle unwritten extents, extent depths, large inode sizes, feature flags and journal sequencing correctly.

Ext4 is not a general copy-on-write snapshot filesystem. The format does not inherently supply a ZFS-style pool or Btrfs subvolume model. Online resizing, fscrypt and quota features are dependent on exact kernel/tool support and format flags; their presence cannot be assumed for an arbitrary ext4 image.

EXT4 - fourth extended filesystem

INFILTRATOROS INTEGRATION BOUNDARY

The InfiltratorOS EXT4.so module is a format interpreter intended for application-level consumers. Shared code with EXT3 does not turn the two formats into identical on-disk structures. The Linux kernel adapter’s supported operations, write safety and target kernel compatibility are independent release qualifications.

The definitive sources are the ext4 overview, on-disk layout and journal documents. Every repair or defragmentation operation requires actual format feature validation and a source-specific test matrix; building this guide does not prove any write path is safe.

EXT4 - fourth extended filesystem

SOURCES AND PRIMARY REFERENCES

The documents below are the historical and engineering reference points used for this guide. Where an on-media field or feature flag is ambiguous, consult the original format documentation and its version-specific implementation notes before interpreting or changing media.

1. Linux kernel ext4 high-level on-disk design

https://docs.kernel.org/filesystems/ext4/overview.html

2. Linux kernel ext4 on-disk structures index

https://docs.kernel.org/filesystems/ext4/index.html

3. Linux ext4 JBD2 journal format

https://docs.kernel.org/filesystems/ext4/journal.html

The InfiltratorOS shared-object module, optional kernel adapter, native builder and release-specific tests are a separate implementation. A successful document build or package installation does not qualify physical writes, recovery, kernel mounting or boot.

EXT4 / technical detail

ON-DISK FIELD AND STRUCTURE INVENTORY

Format-specific field names and meanings are summarised below; where applicable the original specification defines exact endianness, offsets, checksums and format-revision limits.

1. Primary superblock: 1024-byte offset and feature flags.
2. Block-group descriptor: inode table, bitmaps and checksums.
3. Extent header: depth, entry count and maximum entries.
4. Extent leaf: logical start, physical start and length.
5. Extent index: key and child-node physical reference.
6. JBD2: big-endian journal records within little-endian ext metadata.
7. HTree: hashed indexed directories when enabled.
8. Unwritten extents: allocated ranges with special read semantics.

EXT4 / technical detail

HISTORICAL FORMAT DEVELOPMENT

Selected design milestones explain why this format acquired its present layout. Dates refer to historical format or implementation changes, not InfiltratorOS releases.

1. 2006: ext4dev begins expanding ext3 architecture.

2. 2008: ext4 reaches stable Linux 2.6.28 inclusion.

3. 2010s: 64-bit, flex groups and checksumming mature.

4. Later: additional encryption, inline data and allocation features extend formats.

Important historical distinction: newly introduced driver features are not necessarily changes to the original on-disk format. Read the version or feature bits rather than assuming current software support defines historic media.

EXT4 / technical detail

FORMAT VALIDATION AND RECOVERY INVARIANTS

A writable implementation must establish all of these conditions before modifying any original-format media. These checks describe the minimum class of invariants, not proof of complete recovery qualification.

1. Check incompatible flags, 64-bit group descriptors and checksums.

2. Distinguish allocated unwritten extents from initialised file data.

3. Validate extent tree ordering, depth and logical coverage.

4. Respect JBD2 transaction ordering and byte order.

5. Check bitmap, inode and directory state as a coherent group.

Forensic principle: use a separate verified image, retain original structures, and report conflicting evidence instead of inventing a new supposedly clean filesystem state.

EXT4 / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Read superblock feature masks before interpreting a volume. Incompatible, read-only-compatible and compatible features have different consequences for safe access.

2. Resolve flexible block-group and 64-bit descriptor layouts from explicit flags; fixed-size classic ext2 descriptor assumptions can misaddress bitmaps or inode tables.

3. Parse extent headers by magic, depth, entry count and capacity. Follow indexed extent nodes until leaf records map logical file blocks to physical ranges.

4. Distinguish initialized extents from unwritten allocated extents; newly reserved space need not expose existing disk contents as file data.

5. Check delayed-allocation effects when interpreting an unclean image. In-memory allocation plans are not necessarily represented by committed on-disk extents.

6. Read HTree-indexed directories through their index records while still validating actual variable-length directory entries.

7. Handle JBD2 journal records in big-endian form even though ext4 metadata is generally little-endian; confusing their byte order breaks recovery.

8. Validate metadata checksums using the correct feature-dependent checksum seed, UUID and structure coverage, not a generic whole-sector CRC.

EXT4 / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Check extent ordering, logical coverage and tree depth. Overlapping physical extents or conflicting ownership must be distinguished from valid shared metadata cases.

2. An extent marked unwritten is not equivalent to a hole: it has allocated storage but returns zeroed logical content until initialized.

3. Verify that the descriptor, bitmap, inode and superblock checksum algorithms match the exact enabled feature set before declaring corruption.

4. Journal replay must honour transaction sequence, commits, revokes and checksum versions. Arbitrary byte-order substitution is unsafe.

5. Large-file, huge-file, inline-data, bigalloc and flex-group features each alter interpretation; support cannot be inferred from a generic ext4 label.

6. Check compatibility masks before writing to an image produced by a newer kernel. Readability does not imply that unknown state can be preserved by the writer.

7. Recovery should distinguish journal replay, bitmap reconciliation and inode/directory repair rather than treating them as one operation.

8. Record unsupported filesystem revisions or feature combinations explicitly; silently discarding unknown metadata undermines cross-platform compatibility.